



UNIVERSITATEA DIN  
BUCUREȘTI

FACULTATEA DE  
MATEMATICĂ ȘI  
INFORMATICĂ



SPECIALIZAREA TEHNOLOGIA INFORMAȚIEI

Proiect de diplomă

# REZOLVAREA AUTOMATĂ A PUZZLE-URILOR FOLOSIND TEHNICI DE VEDERE ARTIFICIALĂ

Absolvent

Moraru Ilinca-Ana

Coordonator științific

Conf. Dr. Alexe Bogdan

București, iunie 2025

## **Rezumat**

Lucrarea abordează problema rezolvării unui puzzle de dimensiuni variabile (între 100 și 400 de piese) prin metode euristice de identificare a potrivirilor corecte folosind un algoritm iterativ, determinist, ce efectuează deciziile de alăturare sau desfacere. Cele mai bune rezultate au fost obținute prin folosirea euristicilor simple, bazate pe distanțe între caracteristici simple precum culori sau gradienti. Algoritmul de rezolvare a unui puzzle este alcătuit din trei etape, pornind de la alăturări simple de piese în grupuri compacte folosind euristici de potrivire simple, pentru ca, pe măsură ce piesele din puzzle sunt poziționate corect, locurile rămase să fie completate folosind euristici mai complexe. Avantajul acestei abordări, spre deosebire de paradigma populară a rețelelor neurale, este un control amănunțit cu privire la comportamentul programului cât și un timp de execuție mai mic.

## **Abstract**

This paper explores the problem of solving jigsaw puzzles of varying sizes (between 100 and 400 pieces) using heuristic methods to find correct matches. The proposed approach uses a deterministic and iterative algorithm that performs joining or splitting decisions. The best results were achieved with simple heuristics based on distances between basic features like colors and gradients. The algorithm is divided into four stages. It starts by forming compact groups of pieces using basic matching rules, and as more pieces are placed correctly, the remaining spots are filled using complex heuristics. Compared the popular neural network-based methods, this approach offers more control over the algorithm's behavior and requires less computation time.

# Cuprins

|          |                                                                                                                                              |           |
|----------|----------------------------------------------------------------------------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introducere</b>                                                                                                                           | <b>4</b>  |
| <b>2</b> | <b>Potrivirea pieselor folosind distanțe inspirate din vederea artificială</b>                                                               | <b>7</b>  |
| <b>3</b> | <b>Algoritmul de tip Greedy de alăturare a pieselor</b>                                                                                      | <b>16</b> |
| 3.1      | Alăturarea de grupuri . . . . .                                                                                                              | 16        |
| 3.2      | Etapele Algoritmului . . . . .                                                                                                               | 19        |
| 3.2.1    | Etapa I: Alăturarea grupurilor pe baza perechilor de laturi cu o<br>distanță de potrivire mică . . . . .                                     | 19        |
| 3.2.2    | Etapa II: Sistemul de votare . . . . .                                                                                                       | 21        |
| 3.2.3    | Etapa III: Completarea locurilor rămase libere, desfacerea grupu-<br>rilor mici rămase și alăturarea respectivelor piese la grupul principal | 22        |
| 3.3      | Rezultate . . . . .                                                                                                                          | 24        |
| <b>4</b> | <b>Concluzii</b>                                                                                                                             | <b>28</b> |
|          | <b>Bibliografie</b>                                                                                                                          | <b>29</b> |

# Capitolul 1

## Introducere

Problema rezolvării unui puzzle cu un număr mare de piese are o relevanță semnificativă în domeniul vederii artificiale, prin prisma reconstrucției de documente sau picturi murale. Alegerea acestei teme are aplicații practice în diverse domenii, precum cel al arheologiei, unde cercetătorii încearcă să potrivească diverse fragmente de piatră pentru a reconstitui fresce. În acest sens, un proiect de amploare aflat în derulare, RePAIR Project, își propune asamblarea unor fragmente de fresce descoperite în orașul Pompei, cum sunt cele din Figura 1.1. Există peste 10 mii de fragmente ce aparțin de fresce diferite, negrupate în funcție de perețele de care aparțin, astfel încât reasamblarea este aproape imposibilă de realizat doar prin efort uman.

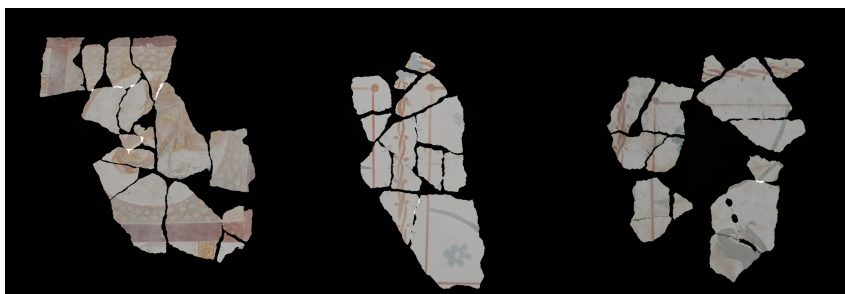
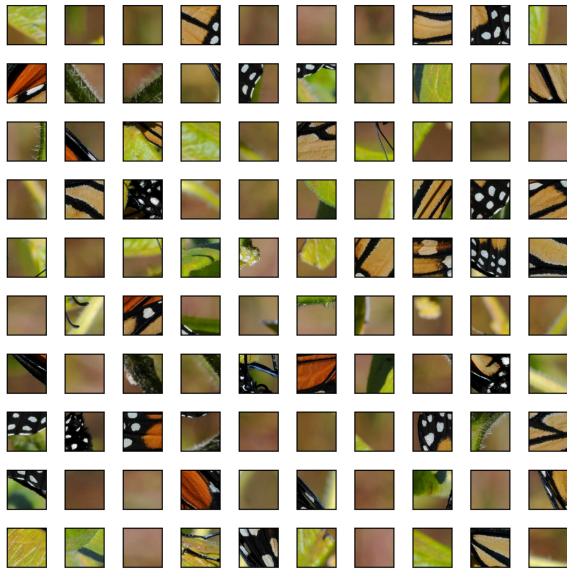


Figura 1.1: **RePAIR Project.** *Exemple de fresce reconstruite de arheologi*

Lucrarea își propune să prezinte o soluție pentru problema rezolvării automate de puzzle-uri cu piese pătrate în care orientarea fiecărei piese în soluția finală nu este cunoscută. Pentru această problemă, considerăm puzzle-uri de dimensiuni ce variază între  $10 \times 10$  și  $20 \times 20$ , deci cu numărul de piese între 100 și 400. Datele de intrare sunt alcătuite din dimensiunile puzzle-ului de rezolvat și piesele corespunzătoare, fiecare dintre ele având o rotație  $\theta \in \{0^\circ, 90^\circ, 180^\circ, 270^\circ\}$ . Piesele formează o mulțime neordonată de obiecte, ordinea lor fiind aleatoare, așa cum se observă și în Figura 1.2. Rezolvarea problemei presupune găsirea pentru fiecare piesă  $P_i$  a tripletului corespunzător  $(x_i, y_i, \theta_i)$  ce reprezintă coordonatele și rotația piesei inițiale în puzzle-ul final reconstruit.



(a)



(b)

Figura 1.2: **Exemplu de puzzle cu  $10 \times 10$  piese.** (a) *Pieseale inițiale, în ordine și cu rotație aleatoare.* (b) *Puzzle-ul rezolvat*

Contribuția majoră a lucrării o constituie propunerea unui algoritm secvențial de rezolvare a unui puzzle, în care alăturarea pieselor se realizează folosind diverse distanțe de potrivire, bazate pe diferențe de culoare, inspirate din vederea artificială. Soluția propusă prezintă alăturarea de piese la nivel de grup, unde în etapa inițială fiecare piesă formează un grup, iar pe măsură ce algoritmul progresează, dimensiunea fiecărui grup crește. Algoritmul prezentat este alcătuit din patru etape:

- i) Identificarea de grupuri de piese alăturate în soluția finală, prin alăturarea de piese ale căror laturi au o distanță de potrivire mică în comparație cu un prag (threshold) ales;
- ii) Alăturarea de grupuri folosind o tehnică de votare;
- iii) Identificarea grupurilor care se potrivesc în regiunile libere obținute în etapele anterioare;
- iv) Potrivirea pieselor rămase prin desfacerea grupurilor rămase și alăturarea lor la grupul principal.

Problema rezolvării automate de puzzle-uri a fost intens studiată, existând o literatură vastă în acest sens [1, 3, 4, 5, 6]. În lucrări care abordează, prin algoritmi iterativi, puzzle-uri cu piese pătrate și rotații necunoscute, metodele propuse sunt în general de tip Greedy, precum identificarea minimului local (Hungarian Method) și căutarea simultană a celor patru vecini ai unei piese [1], construirea de subarbori supuși constrângerilor geometrice [3], sau aplicarea constrângerilor de tip Loop Constraints [6].

O parte semnificativă a rezolvării problemei o reprezintă folosirea unei metrice de măsurare a potrivirii dintre două laturi. Printre abordările notabile se numără asocierea scorurilor de culoare cu descriptori de tip Gist [4], sau integrarea diferențelor de culoare cu statistici ale puzzle-ului [2]. În analiza a cinci tipuri de metrice de potrivire, Cho et al. [2] au arătat că diferența simplă de culoare între marginile pieselor prezintă cea mai mare putere de discriminare. Alte abordări notabile includ aplicarea distanței Mahalanobis [3, 5].

## Capitolul 2

# Potrivirea pieselor folosind distanțe inspirate din vederea artificială

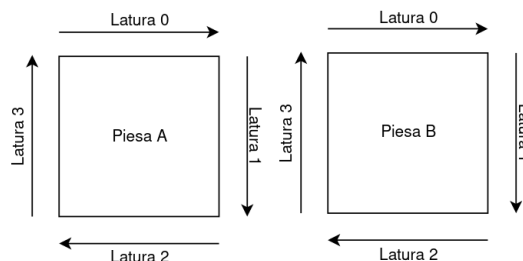


Figura 2.1: Codificarea laturii

Un pas crucial în rezolvarea unui puzzle este găsirea unei măsuri care să cuantifice potrivirea pieselor. O asemenea măsură ar trebui să acționeze ca o funcție de distanță: (i) să aibă valori mici, cât mai apropiate de 0 în cazul perechilor de laturi ale unor piese rotite optim care sunt alăturate în soluția finală; (ii) să aibă valori mari, în jurul valorii 1 pentru perechi de laturi care nu corespund unor piese rotite optime și alăturate în soluția finală. Considerând că găsirea soluției problemei implică determinarea rotației optime a pieselor de puzzle, fiecare latură este stocată în memorie conform unei parcurgeri în direcția acelor de ceasornic, așa cum este ilustrat în Figura 2.1. Pentru a cuantifica potrivirea a două laturi, vom folosi distanțe care utilizează pixelii corespunzători de pe cele două laturi, dar având în vedere următorul aspect important: pixelii de pe o latură vor fi considerați în ordinea normală a stocării lor în memorie, iar pixelii de pe cea de-a doua latură vor fi considerați în ordinea inversă a stocării lor. Astfel, simulăm modul în care cele două laturi ar fi alăturate în realitate.

Pentru măsurarea potrivirii a două piese, calculăm potrivirea între toate perechile corespunzătoare de laturi ale pieselor, obținând în total un număr de 16 distanțe. Putem considera următoarele perechi de laturi:

$$(l_A, l_B) \in \{(l_0^A, l_0^B), (l_0^A, l_1^B), (l_0^A, l_2^B), (l_0^A, l_3^B), \\ (l_1^A, l_0^B), (l_1^A, l_1^B), (l_1^A, l_2^B), (l_1^A, l_3^B), \\ (l_2^A, l_0^B), (l_2^A, l_1^B), (l_2^A, l_2^B), (l_2^A, l_3^B), \\ (l_3^A, l_0^B), (l_3^A, l_1^B), (l_3^A, l_2^B), (l_3^A, l_3^B)\}$$

Calculul distanțelor între toate perechile de laturi corespunzătoare se execută o singură dată, la începutul algoritmului, pentru ca, mai apoi, în funcție de alăturările de piese executate, variabila de rotație a fiecărei piese,  $\theta \in \{0, 1, 2, 3\}$ , să își schimbe valoarea.

În continuare, perechile de forma  $(l_A, l_B)$ , de lungime  $n$ , sunt folosite cu semnificația unei perechi de laturi care nu aparțin aceleiași piese, având forma:

$$l_X = (v_X^1, v_X^2, \dots, v_X^n),$$

unde  $v_X^i$  reprezintă o caracteristică a pixelului  $i$  al laturii  $X$  și care de obicei este considerată a fi tripletul valorilor RGB:

$$v_X^i = (v_{X_R}^i, v_{X_G}^i, v_{X_B}^i).$$

Prezentăm în cele ce urmează diferite distanțe pe care algoritmul dezvoltat le folosește pentru a găsi potrivirea pieselor.

**Distanța la nivel de culoare dintre două laturi.** Măsurăm variația de culoare între laturile  $l_A$  și  $l_B$  a două piese calculând diferențele de culoare RGB între pixelii  $v_A^i$  și  $v_B^i$  de pe cele două laturi. Ecuația 2.1 definește această măsură:

$$d_{\text{culoare}}(l_A, l_B) = \frac{1}{n} \sum_{i=1}^n \sigma \left( \frac{4 \|v_A^i - v_B^{n-i+1}\|_2 - 2}{2} + 0.5 \right), \quad \text{unde } \sigma(x) = \frac{1}{1 + e^{-x}} \quad (2.1)$$



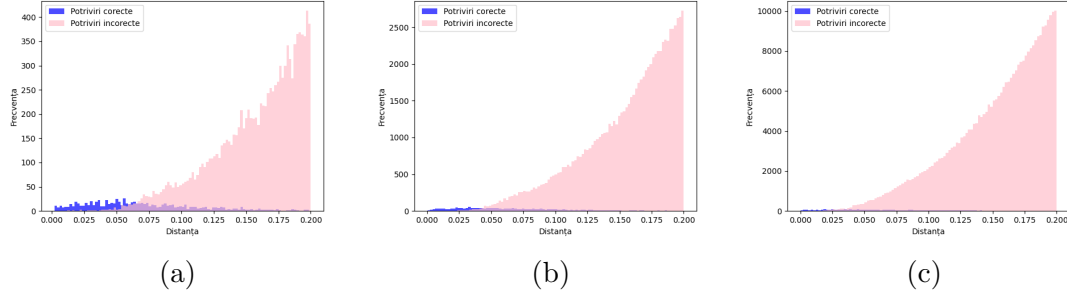


Figura 2.2: Distribuția *distanțelor la nivel de culoare* a perechilor de laturi alăturate (albastru) și nealăturate (roz) în soluția finală pe 10 puzzle-uri. (a) Puzzle-uri cu  $10 \times 10$  de piese. (b) Puzzle-uri cu  $15 \times 15$  de piese. (c) Puzzle-uri cu  $20 \times 20$  de piese.

Deși pentru majoritatea perechilor de laturi ce corespund unor piese alăturate în soluția finală, valorile distanțelor la nivel de culoare se apropie de 0, separarea de perechile de laturi ce corespund unor piese nealăturate în soluția finală este dificilă, așa cum se observă în Figura 2.2. Rezultatele sunt sensibil mai bune folosind canalele RGB față de o conversie în grayscale a normei pixelilor.

**Distanța la nivel de gradient.** Sunt măsurate diferențele de magnitudine ale gradientilor corespunzători pixelilor  $i$  ai laturilor  $l_A$  și  $l_B$  prin scăderea dintre  $\|\nabla G_{l_A}^i\|_2$  și  $\|\nabla G_{l_B}^{n-i+1}\|_2$ , această distanță fiind definită în Ecuația 2.2. **Gradientii** sunt obținuți în urma aplicării pe imaginea inițială a unui filtru Gauss de dimensiune  $5 \times 5$  și a două filtre Sobel de dimensiune  $5 \times 5$ , ce măsoară variațiile de culoare pe axele  $Ox$  și  $Oy$ . Astfel, există, în total, 6 vectori de gradienti ce descriu fiecare canal de culoare RGB pe direcțiile axelor  $Ox$  și  $Oy$  pentru fiecare dintre cele două laturi. **Magnitudinile gradientilor** pentru fiecare pixel  $i$ ,  $\|\nabla G^i\|_2$ , sunt calculate conform Ecuației 2.3.

$$d_{\text{gradient } l_A l_B} = \frac{1}{n} \sum_{i=1}^n \sigma\left(\frac{4|\|\nabla G_{l_A}^i\|_2 - \|\nabla G_{l_B}^{n-i+1}\|_2| - 2}{2} + 0.5\right) \quad (2.2)$$

$$\begin{aligned} \|\nabla G^i\|_2 &= \sqrt{G_x^{i2} + G_y^{i2}}, \quad \text{unde } \nabla G = (G_x, G_y) \\ G_x^i &= \sqrt{G_{x,R}^{i2} + G_{x,G}^{i2} + G_{x,B}^{i2}} \quad G_y^i = \sqrt{G_{y,R}^{i2} + G_{y,G}^{i2} + G_{y,B}^{i2}} \end{aligned} \quad (2.3)$$

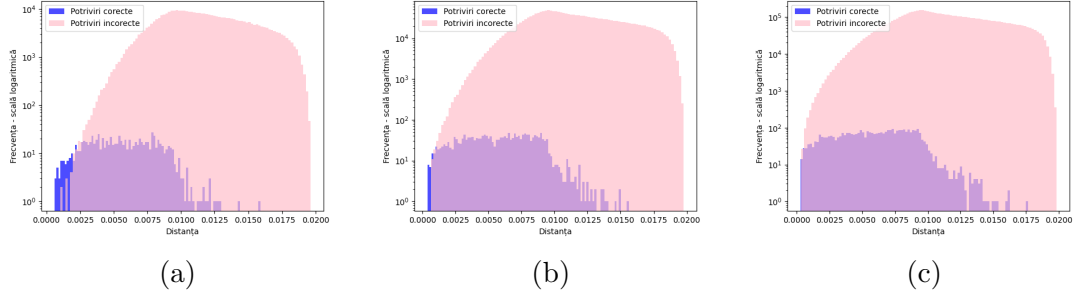


Figura 2.3: Distribuția *distanțelor la nivel de gradient* a perechilor de laturi alăturate (albastru) și nealăturate (roz) în soluția finală pe 10 puzzle-uri, scală logaritmică. (a) Puzzle-uri cu  $10 \times 10$  de piese. (b) Puzzle-uri cu  $15 \times 15$  de piese. (c) Puzzle-uri cu  $20 \times 20$  de piese.

Deși, intuitiv, *distanța la nivel de gradient* ar oferi o informație mai complexă, puterea sa de separabilitate este mai slabă decât *distanța la nivel de culoare*. După cum reiese din Figura 2.3, există doar cazuri izolate, în jurul valorii 0, în care valoarea *distanței la nivel de gradient* pentru acele perechi de laturi ale căror piese sunt alăturate în soluția finală nu este suprapusă cu valorile obținute de perechile de piese ale căror laturi nu sunt alăturate în soluția finală. Contextul culorii propriu-zise dintre pixelii  $v_A^i$  și  $v_B^i$  lipsește, iar simpla magnitudine a variației de culori în cadrul celor două piese poate coincide, fără ca valorile RGB să fie apropiate.

**Suma gradientelor a două laturi.** Cuantificăm gradul total de detalii prezent în laturile  $l_A$  și  $l_B$  prin însumarea magnitudinilor gradientelor  $\|\nabla G_{l_A}^i\|_2$  și  $\|\nabla G_{l_B}^i\|_2$  pentru fiecare pixel  $i$  folosind Ecuația 2.4:

$$SG(l_A l_B) = \sum_{i=1}^n \left[ \sigma\left(\frac{4\|\nabla G_{l_A}^i\|_2 - 2}{2} + 0.5\right) + \sigma\left(\frac{4\|\nabla G_{l_B}^{n-i+1}\|_2 - 2}{2} + 0.5\right) \right] \quad (2.4)$$

Această metrică nu poate clasifica perechile de laturi ca fiind sau nu alăturate în soluția finală, dar, prin folosirea alături de *distanța la nivel de culoare*, are loc o îmbunătățire a performanței.

**Distanța la nivel de culoare normalizată.** Această distanță este rezultatul împărțirii *distanței la nivel de culoare* a două laturi  $l_A$  și  $l_B$  la *suma gradientelor* lor  $SG(l_A l_B)$ , conform Ecuației 2.5. *Distanța la nivel de culoare normalizată* va fi cu atât mai mică cu cât laturile au mai multe detalii, fiind inspirată de strategia umană de rezolvare a unui puzzle, în care primele piese alăturate sunt cele cu detalii semnificative.

$$d_{\text{culoare}}^{\text{normalizată}}(l_A l_B) = \frac{d_{\text{culoare}}(l_A l_B)}{SG(l_A l_B)} \quad (2.5)$$

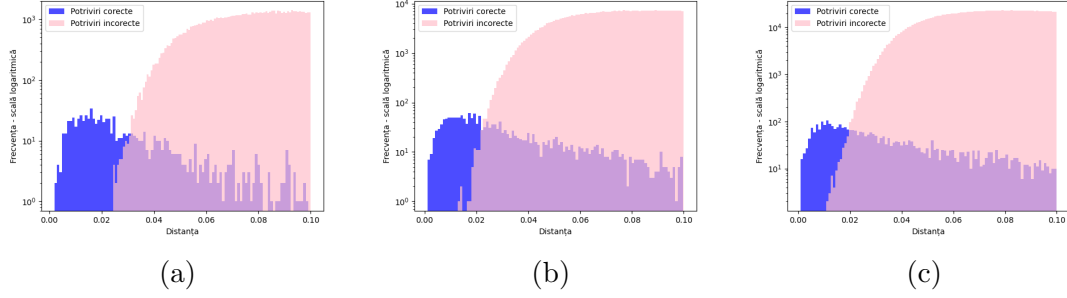


Figura 2.4: Distribuția *distanțelor de culoare normalizată* a perechilor de laturi alăturate (albastru) și nealăturate (roz) în soluția finală pe 10 puzzle-uri, scală logaritmică. (a) Puzzle-uri cu  $10 \times 10$  de piese. (b) Puzzle-uri cu  $15 \times 15$  de piese. (c) Puzzle-uri cu  $20 \times 20$  de piese.

Această distanță ar putea fi folosită în prima etapă a algoritmului, puterea de a separa perechi de laturi care se potrivesc de cele care nu se potrivesc fiind îmbunătățită, așa cum se observă în Figura 2.4. Sunt descurajate perechile de laturi relativ netede, fără gradienti, dar cu nuanțe apropiate, fiind în același timp alese acele perechi cu diferențe de culori mici și cu multe detalii. Folosind un prag pentru fiecare pereche de laturi de  $0.08$  și o medie de  $0.02$  pentru toate alăturările determinate de lipirea a două grupuri de piese, există șanse reduse de eroare. Pentru puzzle-uri de până în  $14 \times 14$  piese, separarea perechilor de laturi care se potrivesc de cele care nu se potrivesc este crescută și rămân sub 10 piese nealăturate grupului principal. Cu toate acestea, *distanța de culoare normalizată* nu este suficient de bună pentru puzzle-uri mai complexe, existând câteva greșeli.

**Distanța la nivel de culoare normalizată îmbunătățită.** Este determinat numărul  $k$  ce reprezintă câți pixeli  $i$  ai laturilor  $l_A$  și  $l_B$  există, pentru care  $d_{\text{culoare}}^i(l_A l_B)$  este mai mare decât un prag de  $0.3$ , această valoare fiind considerată o diferență semnificativă. *Distanța la nivel de culoare normalizată îmbunătățită* va crește exponențial în raport cu *Distanța la nivel de culoare normalizată* pentru valori mari ale lui  $k$ . Ecuația 2.6 definește această distanță:

$$d_{\text{culoare}}^{\text{ni}}(l_A l_B) = d_{\text{culoare}}^{\text{normalizată}}(l_A l_B) \cdot 5^k, \quad (2.6)$$

$$\text{unde } k = \text{Card} \{i \in \{1, \dots, n\} \mid d_{\text{culoare}}^i(l_A l_B) > 0.3\}$$

$$\text{și } d_{\text{culoare}}^i(l_A l_B) = \sigma \left( \frac{4 \|v_A^i - v_B^{n-i+1}\|_2 - 2}{2} + 0.5 \right)$$

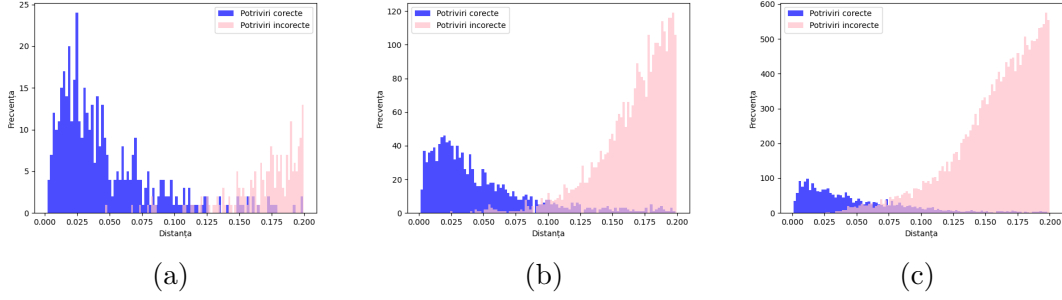


Figura 2.5: Distribuția *distanțelor de culoare normalizată îmbunătățită* a perechilor de laturi alăturate (albastru) și nealăturate (roz) în soluția finală pe 10 puzzle-uri. (a) Puzzle-uri cu  $10 \times 10$  de piese. (b) Puzzle-uri cu  $15 \times 15$  de piese. (c) Puzzle-uri cu  $20 \times 20$  de piese.

Pentru o pereche de laturi a căror piese sunt alăturate în soluția finală,  $k$  va lua valori mici, fiind numărate, de cele mai multe ori, diferențele din apropierea conturilor obiectelor din imaginii. În același timp, diferențe consistente mai mari de 0,3 descalifică multe perechi ale căror laturi nu sunt alăturate în soluția finală. Această distanță este un prim pas în identificarea perechilor de laturi ale căror piese sunt alăturate în soluția finală, în special pentru puzzle-urile mari, unde *distanța de culoare normalizată* produce erori. Numărul de perechi ale căror piese sunt alăturate în soluția finală cu distanța mai mică decât pragul impus se mărește, raportat la *distanța de culoare normalizată*, după cum reiese din Figura 2.5, cu toate că există în continuare imprecizie de ambele părți ale pragului.

**Distanța la nivel de culoare reciprocă.** Pornind de la conceptul de „Best Buddy Score”, folosit în Lucrarea [3], a fost formulată o nouă metrică, numită *Distanța la nivel de culoare reciprocă*. Aceasta măsoară reciprocitatea unei distanțe mici dintre o pereche de laturi, în raport cu cele mai mici distanțe pe care cele două laturi le au. Pentru fiecare dintre cele două laturi  $l_A$  și  $l_B$  sunt calculate acele laturi  $l_C$  și  $l_D$  pentru care  $d_{culoare}^{normalizată}(l_A, l_C)$  și  $d_{culoare}^{normalizată}(l_B, l_D)$  sunt minime. *Distanța la nivel de culoare normalizată* a perechii de laturi  $l_A$  și  $l_B$  este apoi împărțită la aceste valori minime conform Ecuației 2.7:

$$d_{culoare}^{reciprocă} l_A l_B = \frac{d_{culoare}^{normalizată}(l_A l_B)}{\min_{l_C \in \mathcal{L}_{aturi}} d_{culoare}^{normalizată}(l_A l_C)} + \frac{d_{culoare}^{normalizată}(l_A l_B)}{\min_{l_D \in \mathcal{L}_{aturi}} d_{culoare}^{normalizată}(l_B l_D)} - 1 \quad (2.7)$$

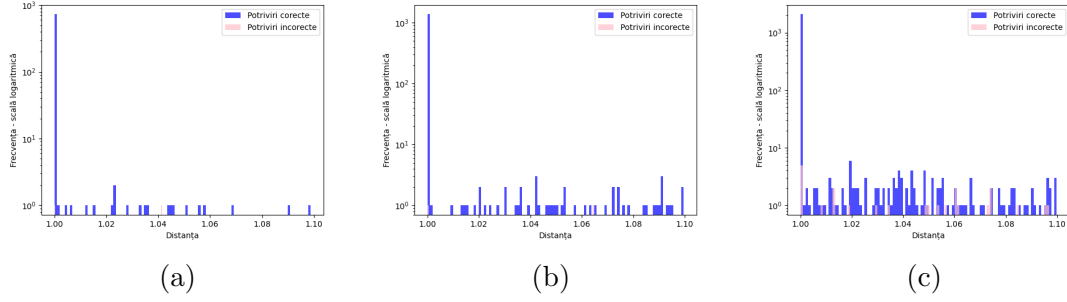


Figura 2.6: Distribuția *distanțelor de culoare reciprocă* a perechilor de laturi alăturate (albastru) și nealăturate (roz) în soluția finală pe 10 puzzle-uri, scală logaritmică. (a) Puzzle-uri cu  $10 \times 10$  de piese. (b) Puzzle-uri cu  $15 \times 15$  de piese. (c) Puzzle-uri cu  $20 \times 20$  de piese.

Dacă latura  $l_C$  găsită este chiar  $l_B$  și latura  $l_D$  găsită este chiar  $l_A$ , cele două laturi  $l_A$  și  $l_B$  vor avea o reciprocitate puternică, iar *distanța la nivelul culorii reciproce* dintre cele două va avea valoarea 1 (valoarea minimă posibilă a acestei metrici). În caz contrar, cu cât *distanțele la nivelul culorii normalizate* ale perechilor  $(l_A, l_C)$  și  $(l_B, l_D)$  sunt mai mici, cu atât *distanța la nivelul culorii reciproce* a perechii  $(l_A, l_B)$  va fi mai mare. Există perechi de laturi  $(l_A, l_B)$  pentru care  $l_C = l_B$ , dar  $l_D \neq l_A$ , iar *distanța la nivel de culoare reciprocă* pune la îndoială precizia *distanței la nivel de culoare normalizată* a unor astfel de perechi.

Figura 2.7a ilustrează toate *distanțele la nivel de culoare normalizate* pe care le are o anumită latură  $l_X$  cu toate celelalte laturi ale puzzle-ului (stânga) și *distanțele la nivel de culoare reciproce* (dreapta), perechea de laturi a căror piese sunt alăturate în soluția finală  $(l_X, l_Y)$  fiind marcată cu albastru. Există 12 perechi de laturi cu *distanțe la nivel de culoare normalizate* mai mici decât perechea  $(l_X, l_Y)$ , dar prin raportarea la *distanțele mari la nivel de culoare normalizate* ale laturii  $l_Y$ , perechea  $(l_X, l_Y)$  se diferențiază ca având cea mai mică *distanță la nivel de culoare reciprocă*. Figura 2.7b descrie *distanțele* unei laturi  $l_X$  care se află pe marginea soluției rezolvate, deci care nu este, în final, alăturată niciunei alte piese. Cu toate că o parte din *distanțele la nivelul culorii normalizate* sunt suficient de mici pentru a fi considerate alăturate în soluția finală, acestea sunt mari în comparație cu distribuțiile acelor laturi. Prin urmare, *distanțele la nivelul culorii normalizate* ale acelor perechi de laturi au valori mari, de peste 3, primind astfel o prioritate redusă în rezolvarea puzzle-ului.

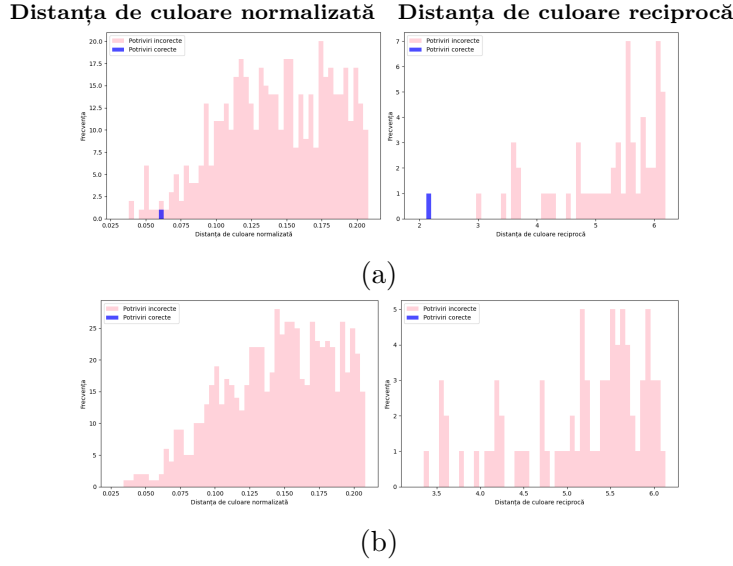


Figura 2.7: **Exemplu de distribuție a distanțelor a două laturi.** (a) *Distanțele de culoare normalizate și distanțele de culoare reciprocă a unei laturi poziționată din interiorul soluției finale.* (b) *Distanțele de culoare normalizate și distanțele de culoare reciprocă a unei laturi poziționate pe marginea soluției finale.*

Această distanță se bazează pe prezumțiile anterioare făcute de *distanța la nivel de culoare normalizată*, realizându-se în același timp o normalizare în raport perechile  $l_A, l_C$  și  $l_B, l_D$ . Puterea de discriminare este mare, după cum arată Figura 2.6. Totuși, există și perechi unde ambele laturi sunt poziționate pe marginea soluției finale, ce nu ar trebui alăturate niciunei alte laturi, unde *distanța la nivel de culoare reciprocă* este minimă, în special dacă acele laturi au culori asemănătoare și puține detalii (ex. cer senin). Prin urmare, *distanța la nivel de culoare reciprocă* nu va fi folosită de sine stătătoare, fiind, în același timp, un criteriu de luat în seamă, oferind un context mai larg asupra probabilităților de potrivire.

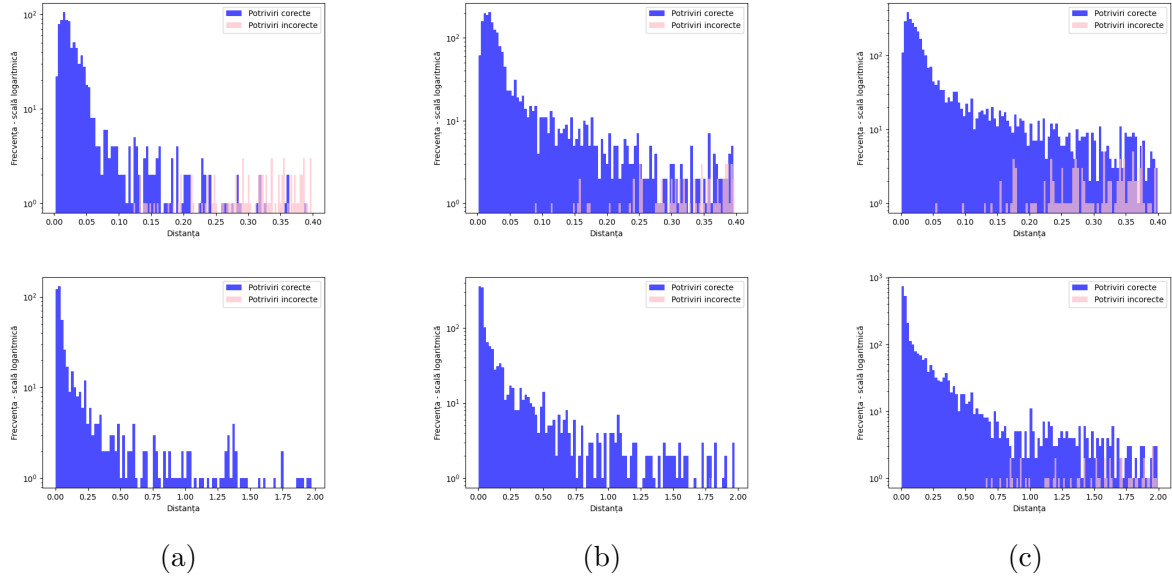


Figura 2.8: Distribuția *distanțelor de culoare reciprocă înmulțită cu alte distanțe* pentru perechi de laturi alăturate (albastru) și nealăturate (roz), în soluția finală pe 10 puzzle-uri, scală logaritmică. (1) *Distanța de culoare reciprocă · distanța de culoare normalizată*. (2) *Distanța de culoare reciprocă · distanța de culoare normalizată îmbunătățită*. (a) Puzzle-uri cu  $10 \times 10$  de piese. (b) Puzzle-uri cu  $15 \times 15$  de piese. (c) Puzzle-uri cu  $20 \times 20$  de piese.

Înmulțind *distanța de culoare reciprocă* cu *distanța de culoare normalizată* sau cu *distanța de culoare normalizată îmbunătățită*, se pot diferenția perechile de laturi alăturate în soluția finală de perechile de laturi nealăturate în soluția finală, cel de-al doilea produs având mai multe perechi identificate, după cum observă în Figura 2.8. Cu toate acestea, există perechi de laturi alăturate în soluția finală ce au distanțe mai mari decât de pragul de discriminare. Prin urmare, aceste două distanțe sunt potrivite pentru un pas incipient al rezolvării, unde este dorită identificarea a cât mai multe perechi de laturi alăturate în soluția finală, fără riscul identificării eronate, fiind admise omiteri a anumite perechi de laturi greu de identificat, ale căror piese sunt alăturate în soluția finală. Aceste înmulțiri valorifică avantajele celor două distanțe, oferind în final mai multe perechi identificate ca alăturate în soluția finală decât cele două individual, cu o precizie mai mare.

## Capitolul 3

# Algoritmul de tip Greedy de alăturare a pieselor

### 3.1 Alăturarea de grupuri

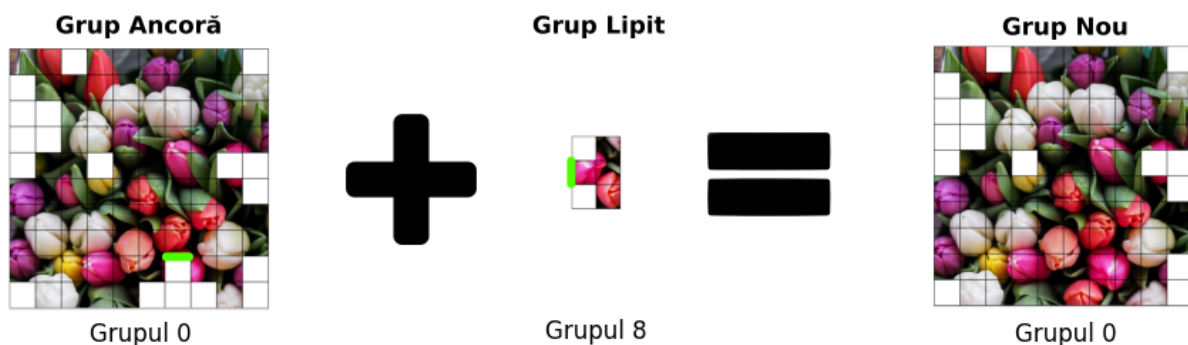


Figura 3.1: Exemplu de alăturare a două grupuri  $G_a$  și  $G_l$ .

Într-un puzzle de dimensiune  $N \times N$ , fiecare *grup* este alcătuit din una sau mai multe *piese*  $P_i$ , ale căror poziții și rotații sunt descrise de tripletul corespunzător:

$$P_i = (x_i, y_i, \theta_i), \quad \text{unde } 0 \leq x_i, y_i < N, \quad \theta_i \in \mathbb{Z}_4.$$

Fiecare latură  $L_i$  a unei piese  $P_i$  are asociat un  $id_i \in \mathbb{Z}_4$ , ce îi corespunde în mod unic. Atribuirea de Id-uri la nivelul unei piese are loc la începutul algoritmului, printr-o parcurgere în ordinea acelor de ceasornic, având codificarea 0 – sus, 1 – dreapta, 2 – jos, 3 – stânga, iar acestea nu suferă ulterior modificări pe parcursul algoritmului. Fiecare latură  $L_i$  are asociată o orientare, numită *orientare<sub>i</sub>*  $\in \mathbb{Z}_4$ , ce reprezintă rezultatul adunării  $id_i$  cu rotația pe care piesa  $P_i$  o execută la alăturarea la grupul de care aparține și exprimă orientarea „reală” a laturii, având codificarea 0 – sus, 1 – dreapta, 2 – jos, 3 – stânga.

Pentru alăturarea a două grupuri, vom considera în continuare:



- i) *grupul ancoră*  $G_a$ , unde  $\forall P_i \in G_a$ , valorile  $(x_i, y_i, \theta_i)$  și  $(id_i, orientare_i)$  rămân nemodificate;
- ii) *grupul lipit*  $G_l$ , care va fi transformat în  $G_l^*$  în urma aplicării rotației și translației corespunzătoare, astfel încât:

$$\begin{aligned} P_i &= (x_i, y_i, \theta_i) \rightarrow P_i^* = (x_i^*, y_i^*, \theta_i^*), \\ L_j &= (id_i, orientare_i) \rightarrow L_j^* = (id_i, orientare_i^*), \quad \forall P_i \in G_l, \quad \forall L_j \in Laturi(P_i). \end{aligned}$$

Două grupuri  $G_a$  și  $G_l$  pot fi alăturate doar dacă sunt îndeplinite următoarele condiții:

$$\begin{aligned} (x_i, y_i) &\neq (x_j, y_j), \quad \forall P_i = (x_i, y_i, \theta_i) \in G_a, \quad \forall P_j = (x_j, y_j, \theta_j) \in G_l^* \\ 0 &\leq x_k, y_k < N, \quad \forall P_k = (x_k, y_k, \theta_k) \in G_a \cup G_l^* \end{aligned} \quad (3.1)$$

Alăturarea grupurilor  $G_a$  și  $G_l$  este determinată de o distanță mică între două laturi, numite  $L_a$  și  $L_l$ , corespunzătoare a două piese, numite în continuare  $P_a$  și  $P_l$  ce aparțin grupurilor  $G_a$  și  $G_l$ . Pentru verificarea condițiilor de mai sus în alăturarea a două grupuri  $G_a$  și  $G_l$ , calculăm un triplet  $(\nabla_x, \nabla_y, \nabla_\theta)$  cu ajutorul cărora vom transla și roti grupul  $G_l$ , astfel simulând alăturarea lor.

În continuare, determinarea tripletului  $(\nabla_x, \nabla_y, \nabla_\theta)$  este explicată și exemplificată.

Sunt calculate orientările  $orientare_a$  și  $orientare_l$  ale laturilor  $L_a$  și  $L_l$  față de grupurile  $G_a$  și  $G_l$ , de care aparțin, folosindu-se formula:

$$orientare_{L_x} = id_{L_x} + \theta_{P_x} \quad (3.2)$$

Figura 3.1 ilustrează un exemplu de alăturare de grupuri bazată pe o potrivire bună a două laturi,  $L_a$  și  $L_l$ , marcate cu verde. Acestea au  $id_a$  și  $id_l$  egale cu 1 și 0, iar piesele de care aparțin,  $P_a$  și  $P_l$ , au rotațiile  $\theta_a = 1$  și  $\theta_l = 3$ . Aplicând Ecuația 3.2, se obțin  $orientare_a = 2$  și  $orientare_l = 3$ , după cum descrie Figura 3.2a.

$(orientare_a, orientare_l)$  corespunzătoare perechii de laturi  $L_a$  și  $L_l$  va fi, după alăturare, una din următoarele perechi de *orientări*: (dreapta, stânga), (stânga, dreapta), (jos, sus), (sus, jos), codificate  $Perechi\_Orientări \in \{(1, 3), (3, 1), (2, 0), (0, 2)\}$ . Prin urmare,  $orientare_l^*$  își va schimba valoarea în funcție de  $orientare_a$ , astfel încât  $(orientare_a, orientare_l^*) \in Perechi\_Orientări$ . În continuare, este determinată rotația  $\nabla_\theta$  pe care grupul  $G_a$  o va face pentru alăturarea grupurilor  $G_a$  și  $G_l$  conform Ecuației 3.3:

$$\nabla_\theta = orientare_l^* - orientare_l \quad (3.3)$$

În figura 3.2b, este exemplificată rotația  $\nabla_\theta$  pe care o va executa grupul  $G_l$ . Conform valorii  $orientare_a = 2$ ,  $orientare_l^*$  devine 0, iar aplicând Ecuația 3.3, rotația aplicată

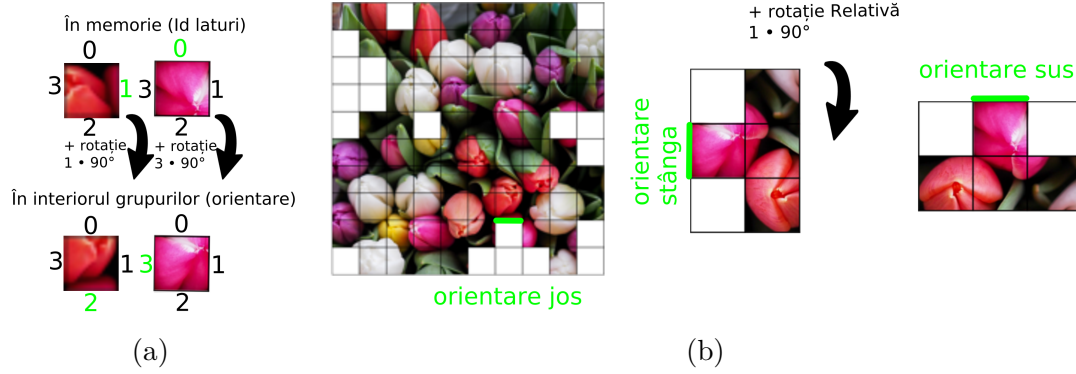


Figura 3.2: **Rotirea grupului  $G_l$ .** (a) *Calcularea orientărilor  $orientare_a$  și  $orientare_l$  ale laturilor  $L_a$  și  $L_l$ .* (b) *Rotirea grupului  $G_l$  în funcție de orientările  $orientare_a$  și  $orientare_l$  ale laturilor  $L_a$  și  $L_l$*

asupra grupului  $G_l$  este  $\nabla_\theta = 1$ . Astfel, grupul  $G_l$  execută o rotație de  $90^\circ$ .

Următoarea etapă presupune determinarea translațiilor  $\nabla_x$  și  $\nabla_y$  pe axele  $Ox$  și  $Oy$  ale grupului  $G_l$ . Acestea se calculează pe baza coordonatelor  $x_a, y_a, x_l, y_l$  ale pieselor  $P_a$  și  $P_l$ , precum și în funcție de *diferență relativă*  $\delta = (\delta_x, \delta_y)$ , determinat în funcție de perechea de orientări  $(orientare_a, orientare_l^*)$ . *Diferența relativă*  $\delta$  este definită astfel:

$$(\delta_x, \delta_y) = \begin{cases} (1, 0), & \text{dacă } (orientare_a, orientare_l^*) = (1, 3) \\ (-1, 0), & \text{dacă } (orientare_a, orientare_l^*) = (3, 1) \\ (0, 1), & \text{dacă } (orientare_a, orientare_l^*) = (2, 0) \\ (0, -1), & \text{dacă } (orientare_a, orientare_l^*) = (0, 2) \end{cases} \quad (3.4)$$

Translațiile  $\nabla_x$  și  $\nabla_y$  se determină conform Ecuației 3.5:

$$\begin{aligned} \nabla_x &= x_a + \delta_x - x_l \\ \nabla_y &= y_a + \delta_y - y_l \end{aligned} \quad (3.5)$$

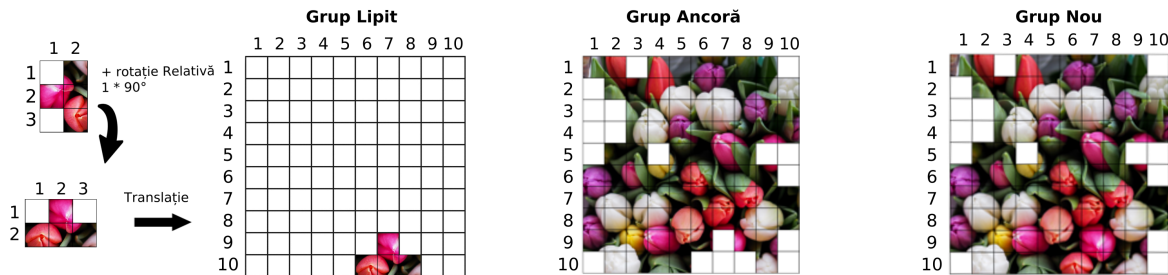


Figura 3.3: **Translarea grupului lipit și unirea celor două grupuri**

În Figura 3.3, este exemplificată translația grupului  $G_l$ , după executarea rotației. Piesa  $P_a$  are valorile  $x_a = 7$  și  $y_a = 8$ , iar piesa  $P_l$  are valorile  $x_l = 2$  și  $y_l = 1$ . Fiind vorba de o pereche de laturi orientate (jos, sus), conform Ecuației 3.5,  $(\delta_x, \delta_y)$  va fi  $(0, 1)$ . Aplicând Ecuația 3.5, se obțin  $\nabla_x = 5$  și  $\nabla_y = 8$ . Astfel, fiecare piesă  $P_i$  din grupul  $G_l$  va aduna la coordonatele sale  $x_i, y_i$  valorile  $\nabla_x = 5$  și  $\nabla_y = 8$ . De exemplu, piesa  $P_l$ , de coordonate  $(2, 1)$ , va avea poziția  $(7, 9)$ .

## 3.2 Etapele Algoritmului

Algoritmul implică ordonarea optimă a alăturărilor executate, pentru ca puzzle-ul să fie rezolvat cât mai aproape de soluția finală. Pentru a fi realizată această ordonare, sunt folosite diverse strategii cât și distanțe prezentate în Capitolul 2. Abordarea algoritmului este de tip Greedy, întrucât, în cazul problemei rezolvării unui puzzle, prin identificarea optimelor locale - alăturări de grupuri regăsite în soluția finală, se ajunge, de obicei, la optimul global - soluție corectă. Totodată, abordarea de tip exhaustivă întâmpină dificultăți din punct de vedere computațional.

În prima etapă, puzzle-ul este simplificat, prin realizarea acelor alăturări pentru care laturile corespunzătoare au distanțele suficient de mici pentru a fi considerate ca fiind alăturate în soluția finală. În a doua etapă, alăturările se realizează în urma unui sistem de vot. A treia etapă presupune identificarea acelor grupuri care se potrivesc cel mai bine în spațiile rămase libere, ca apoi, în cazul în care soluția nu converge la un singur grup, acele grupuri rămase sunt desfăcute și alăturate la grupul principal.

### 3.2.1 Etapa I: Alăturarea grupurilor pe baza perechilor de laturi cu o distanță de potrivire mică

Fiecare piesă pornește ca fiind un grup de sine stătător și este executată o căutare pe toate perechile de laturi, sortate crescător după *distanța la nivel de culoare normalizată*. Este verificat dacă fiecare nouă alăturare de laturi  $L_a$  și  $L_l$  respectă o condiție individuală impusă. Dacă unirea grupurilor respectă condițiile, unul dintre grupuri este desființat, iar celălalt este actualizat ca fiind rezultatul alăturării celor două grupuri.

Condiția individuală *Condiție I* folosită în primă fază în această etapă este:

$$d_{\text{culoare}}^{\text{ni}}(L_a, L_l) \cdot d_{\text{culoare}}^{\text{reciprocă}}(L_a, L_l) < th_1. \quad (3.6)$$

Pragul  $th_1$  este determinat experimental ca fiind  $0.6$ . Motivația alegerii acestei euristici este aceea că, în începutul rezolvării se dorește identificarea a cât mai multor perechi de laturi ale căror piese sunt alăturate în soluția finală, cu certitudinea că nu sunt efectuate alăturări ce nu se regăsesc în soluția finală.

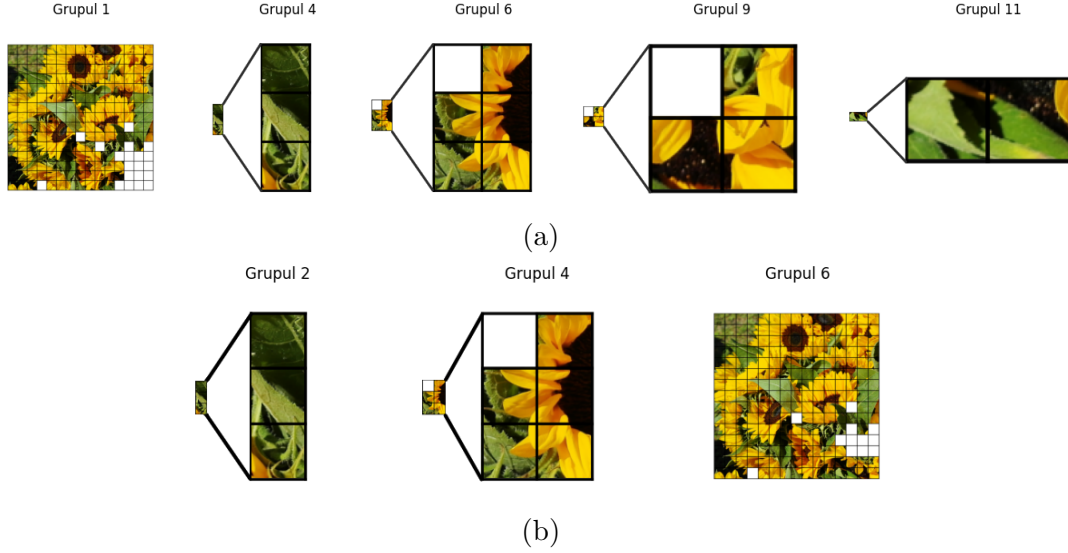


Figura 3.4: **Executarea Etapei I**, exemplu de puzzle cu  $15 \times 15$  piese. (a) *Rezultatul în urma Condiției I.* (b) *Rezultatul în urma Condiției II.*

Această distanță respinge orice pereche de laturi pentru care etichetarea nu este ușoară (există multe diferențe de culoare la nivel de pixel mai mari de  $0.3$  sau perechea  $L_a, L_l$  nu are o reciprocitate puternică).

Cu toate că această respingere restrânge numărul de alăturări ce au loc în această etapă, algoritmul final este mai rapid prin aplicarea acestei strategii. Acest lucru se întâmplă pentru că, plecând de la prezumția că configurațiile grupurilor nou formate se regăsesc în soluția finală, nu este nevoie de identificarea erorilor și executarea de desfaceri ale grupurilor de piese, ci doar de executarea de alăturări optime. După cum este ilustrat în Figura 3.4a, majoritatea alăturărilor necesare rezolvării sunt realizate folosind distanța calculată în Ecuația 3.6.

După ce toate perechile de laturi care îndeplinesc condiția individuală anterioară determină executarea de alăturări, este aplicată aceeași tactică cu o condiție diferită, numită *Condiție II*:

$$E[d_{\text{culoare}}^{\text{normalizată}}(L_a, L_l)] < th_2, \quad \forall L_a \in G_a, L_l \in G_l. \quad (3.7)$$

unde  $L_a$  și  $L_l$  sunt laturile ce devin alăturate în urma alăturării grupurilor de care aparțin, iar  $th_2$  este egal cu  $0.02$ . Această metrică găsește noi perechi de laturi ce determină executarea de alăturări, unde probabilitatea ca acestea să fie alăturate în soluția finală este mare. Impunând un prag al mediei de  $0.02$ , cel puțin jumătate din noile potriviri vor avea o distanță egală sau mai mică de  $0.01$ , distanță ce oferă certitudini cu privire la regăsirea alăturării în soluția finală. Astfel, sunt executate alăturări ale acelor perechi de laturi ce sunt alăturate în soluția finală, dar a căror distanță este prea mare.

### 3.2.2 Etapa II: Sistemul de votare

În această etapă, fiecare latură  $L_a$  de pe marginea grupurilor formate, aparținând grupului  $G_a$ , va căuta acea latură  $L_l$  care se află, și ea, pe marginea grupurilor formate, dar fără a aparține aceluiași grup, astfel încât distanța perechii  $L_a, L_l$  să fie minimă. Votul are forma:

$$v_x = (G_a, G_l, \nabla_x, \nabla_y, \nabla_\theta),$$

unde  $\nabla_x, \nabla_y, \nabla_\theta$  exprimă translația și rotația grupului  $G_l$  pentru alăturarea sa la  $G_a$ .

Condiția folosită în Etapa II este:

$$\begin{aligned} \mathbb{E}[d_{\text{culoare}}^{\text{normalizată}}(L_a, L_l)] &< th_{\mathcal{J}}, \\ \forall L_a \in G_a, L_l \in G_l, L_a, L_l \text{ alăturate.} \end{aligned} \quad (3.8)$$

unde  $th_{\mathcal{J}}$  începe ca având valoarea  $0.024$ , și devine progresiv mai mare, prin înmulțirea cu  $1.2$ , pe măsură ce sunt realizate toate acele alăturări ce respectă pragul anterior. Etapa de votare se încheie atunci când  $th_{\mathcal{J}}$  ajunge la  $1.0$ . La fiecare iterație, sunt numărate toate voturile unde condiția 3.8 este îndeplinită, pe baza cheii unice de forma  $v_x$ , și acea cheie cu cele mai multe voturi determină executarea respectivei alăturări ale grupurilor  $G_a$  și  $G_l$ .

Această strategie reușește să maximizeze contextul existent, alăturarea executată fiind întotdeauna cea cu cele mai multe voturi  $v_x$  dintre cheile de vot cu distanțele dintre laturi cele mai mici. Prin urmare, decizia este influențată de un număr mare de votanți, grupurile cu multe piese primind prioritate, dar numărul de votanți nu este singurul criteriu. O abordare în care singurul criteriu este numărul de voturi  $v_x$  are ca rezultat alăturarea de laturi cu variații de culori mici, cum ar fi cerul senin, fără ca acele alăturări să fie în majoritatea cazurilor regăsite în soluția finală.

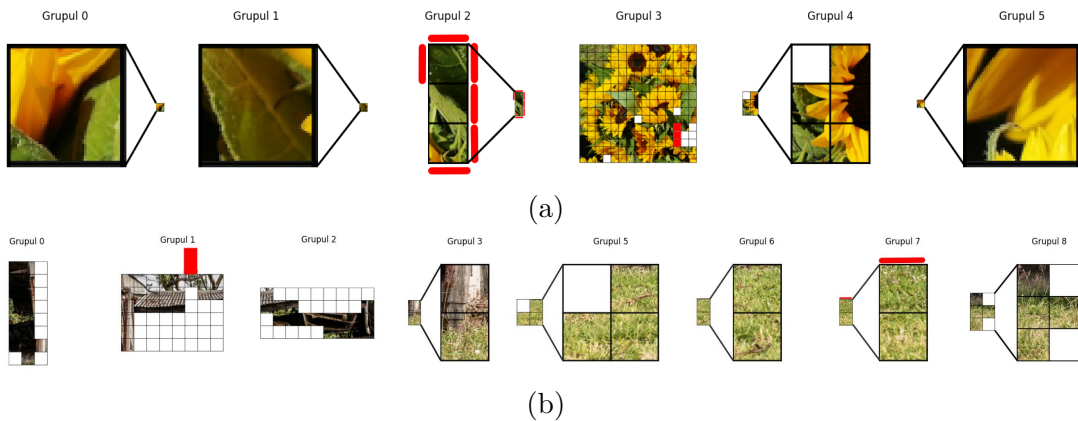


Figura 3.5: **Etapa II**, exemple de puzzle-uri cu  $15 \times 15$  piese. (a) *Exemplu de alăturare de grupuri regăsită în soluția finală.* (b) *Exemplu de alăturare de grupuri ce nu se regește în soluția finală*

În Figura 3.5a este reprezentată o alăturare a două grupuri de piese din Etapa II în care Grupul 2 are 6 muchii (marcate cu linii roșii) care votează alăturarea lor la Grupul 3, cu o translație de  $(12, 10)$  și o rotație de  $0^\circ$ , ce plasează Grupul 2 pe pozițiile colorate cu roșu din Grupul 3. Aceste 6 laturi au avut, fiecare individual, dintre toate voturile posibile, cea mai mică distanță față de laturile corespunzătoare unirii indicate, voturile fiind validate ca fiind posibile și cu o medie acceptată de pragul variabil. În Figura 3.5b este reprezentată o alăturare din etapa de votare a unui alt puzzle, alăturare ce nu se regăsește în soluția finală. Latura din Grupul 7, marcată cu o linie, roșie votează pentru alăturarea cu Grupul 1 cu o translație de  $(5, -3)$  și rotație de  $180^\circ$ . În mulțimea de voturi cu medie  $[0.34, 0.40]$  nu se află decât mutări cu un singur vot, dintre care această mutare are cel mai mic scor.

### 3.2.3 Etapa III: Completarea locurilor rămase libere, desfacerea grupurilor mici rămase și alăturarea respectivelor piese la grupul principal

Pentru fiecare poziție neocupată  $x, y$  a grupurilor formate, sunt numărate câte poziții *vecine* (sus, jos, stânga, dreapta) sunt ocupate, numărul de vecini determinând gradul de constrângere la care este supusă poziția  $x, y$ . Aceste poziții neocupate sunt completate în ordinea gradului lor de constrângere astfel încât media tuturor noilor alăturări de laturi să fie minimă, folosindu-se *distanța la nivel de culoare normalizată*.

Există situații în care algoritmul nu reușește să ajungă la un singur grup de piese, chiar dacă se iterează prin toate perechile de laturi în ordinea distanțelor de potrivire, fără a fi impus un prag. Acest fenomen are loc pentru că acele grupuri rămase nu au forme compatibile. În continuare, este păstrat doar acel grup alcătuit din cele mai multe piese, iar toate celelalte grupuri sunt desfăcute. Sunt apoi efectuate alăturări folosind aceeași strategie și același criteriu, *distanța la nivel de culoare normalizată*, cu diferența importantă că, piesele ce provin din grupurile desfăcute nu vor fi alăturate între ele, ci doar grupului principal.

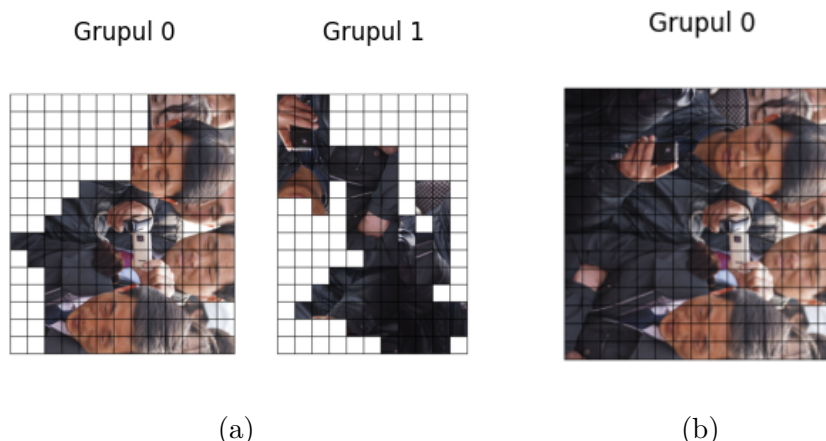


Figura 3.6: **Etapa III**, exemplu de puzzle cu  $15 \times 15$  piese. (a) Rezultatul în urma executării *Etapei II*. (b) Rezultatul în urma executării *Etapei III*.

Figura 3.6 ilustrează o rezolvare în Etapa III, unde se ajunge la două grupuri, al doilea dintre ele având alăturări de piese ce nu se regăsesc în soluția finală, potrivirea fiind dificilă din cauza lipsei de detalii. Acest grup este desfăcut, ulterior fiind găsite locurile potrivite pentru fiecare piesă, astfel că rezultatul corespunde cu soluția finală.

În cele mai multe cazuri, acele piese ce nu au fost alăturate grupului principal în urma etapelor precedente au asociate acele laturi unde identificarea alăturării optime e dificilă. Etapa III are ca scop principal minimizarea numărului de alăturări ce nu se regăsesc în soluția finală, întrucât, existând mai mult de un singur grup după executarea celorlalte etape, suntem siguri că există alăturări ce nu se regăsesc în soluția finală.

Figura 3.7a prezintă un stadiu intermediar al puzzle-ului în cadrul execuției Etapei I, unde există alăturări greșite în cadrul Grupului 6 și Grupului 9. Figura 3.7b reprezintă rezolvarea în urma executării complete a Etapei I, unde Grupul 3, cu alăturări diferite de soluția finală, îi sunt alăturate alte piese, într-un mod diferit de optim. În urma umplerii golurilor în Etapa III, starea puzzle-ului, ilustrată în stânga Figurii 3.7c prezintă diferențe în raport cu soluția finală în ceea ce privește Grupul 1. Acesta este rezultatul unui „efect de domino” ce are loc o dată cu prima alăturare de piese diferită de optim. După executarea Etapei III, numărul de alăturări ce nu se regăsesc în soluția finală scade, dar nu poate fi redus la zero.

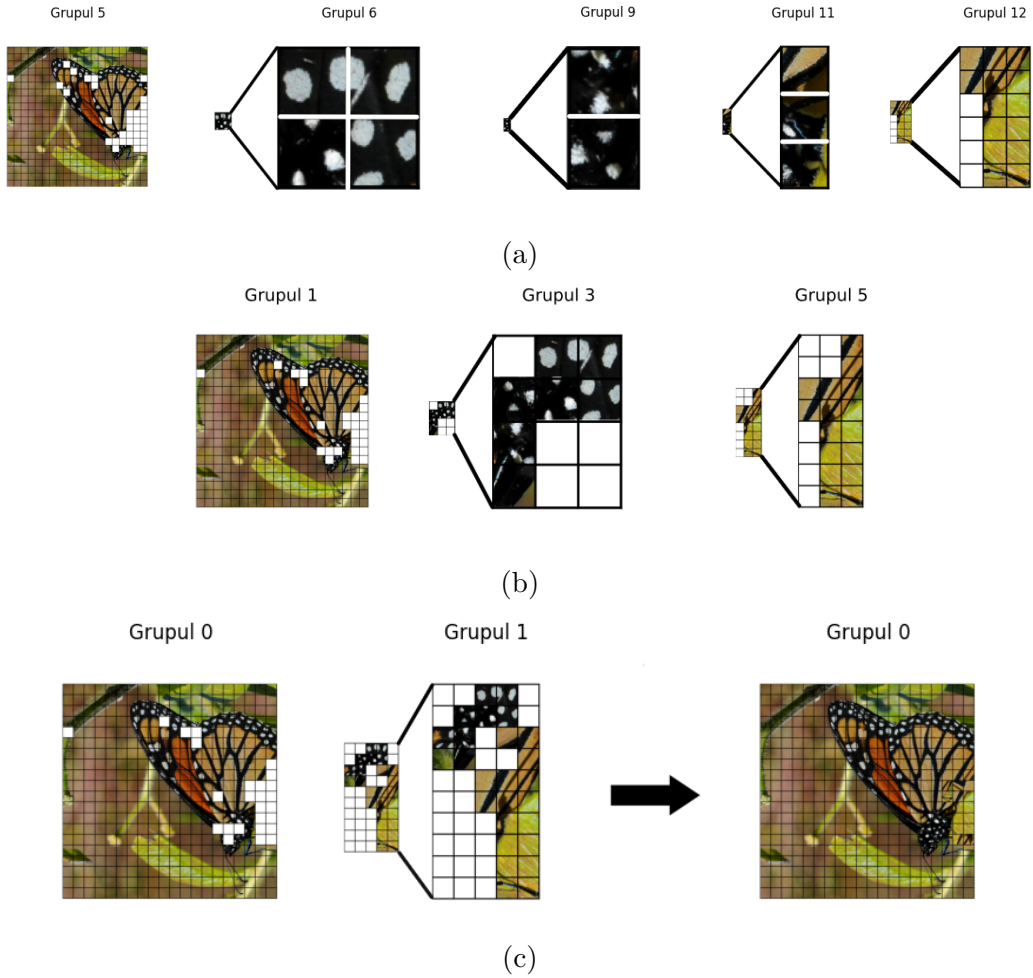


Figura 3.7: **Exemplu al etapelor de rezolvare ale unui puzzle,  $15 \times 15$  piese.** (a) *Stadiu intermediar de rezolvare în Etapa I.* (b) *Rezolvarea în urma Etapei I.* (c) *Umplerea golurilor (stânga), desfacerile grupurilor și alăturările executate în urma Etapei III (dreapta).*

### 3.3 Rezultate

Performanța fiecăreia dintre etape se măsoară pe baza procentajului de perechi de laturi ce sunt alăturate conform cu soluția finală la încheierea executării etapei respective. Această performanță este măsurată pe baza rezolvării a 120 de imagini naturale, de rezoluții variabile, cuprinse între  $800 \times 800$  și  $1400 \times 1400$  de pixeli, extrase din setul de date *DIV2K*. Aceste imagini au fost împărțite în piese pătrate și au fost folosite ca date de intrare cu o ordine și rotație aleatoare, astfel fiind alcătuite puzzle-uri de dimensiuni  $10 \times 10$ ,  $15 \times 15$  și  $20 \times 20$ .

În Tabelul 3.1 sunt prezentate procentajele corespunzătoare fiecărei etape și Figura 3.8 ilustrează distribuția procentului de performanță al fiecărei etape ale algoritmului, pentru puzzle-uri cu  $15 \times 15$  piese.



Tabelul 3.1: Performanțele fiecărei etape de rezolvare a algoritmului

| Dimensiunea Puzzle | Etapa I Criteriu I | Etapa I Criteriu II | Etapa II | Etapa III |
|--------------------|--------------------|---------------------|----------|-----------|
| $10 \times 10$     | 68%                | 78%                 | 94%      | 92%       |
| $15 \times 15$     | 74%                | 81%                 | 92%      | 85%       |
| $20 \times 20$     | 78%                | 82%                 | 89%      | 82%       |

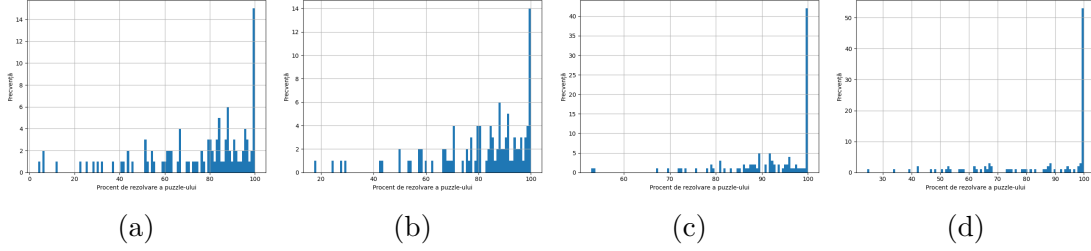


Figura 3.8: Distribuția procentului de laturi alăturate conform soluției finale, puzzle-uri cu  $15 \times 15$  de piese. (a) *Etapa I, Criteriul I*. (b) *Etapa I, Criteriu II*. (c) *Etapa II*. (d) *Etapa III*.

Cele mai multe dintre alăturările de piese ce se regăsesc în soluția finală au loc în Etapa I, unde peste 80% din toate perechile de laturi alăturate sunt conforme cu soluția finală. În Etapa II, se atinge o medie de 91% din totalul de perechi de laturi alăturate conform soluției finale. Etapa III rezolvă puzzle-ul astfel încât acesta să se încadreze în dimensiunile finale ale soluției, astfel că performanța rezolvării nu este îmbunătățită la nivel global, ci chiar scade la 86%, rezultatele variind de la caz la caz.

Odată ce puzzle-ul este terminat, cu toate piesele poziționate, performanța algoritmului de rezolvare poate fi evaluată printr-o *distanță*, unde sunt însumate distanțele euclidiene ale pieselor așezate greșit de către algoritm în raport cu poziția lor ideală. Cu toate că lucrarea nu oferă implementări pentru minimizarea unei asemenea măsuri, ea facilitează compararea algoritmului propus cu rezolvarea umană, unde piesele poziționate diferit de soluția finală tind a fi la o distanță mică de locul lor real. În mod firesc, cu cât puzzle-ul are mai multe piese, aceste distanțe cresc, întrucât piesele poziționate diferit de soluția ideală tind a fi plasate la extremitățile puzzle-ului. Tabelul 3.2 prezintă valorile *distanței* obținute pentru puzzle-uri cu dimensiuni de  $10 \times 10$ ,  $15 \times 15$  și  $20 \times 20$  de piese.

Tabelul 3.2: Distanța

| Dimensiunea Puzzle | Distanța |
|--------------------|----------|
| $10 \times 10$     | 53       |
| $15 \times 15$     | 346      |
| $20 \times 20$     | 826      |

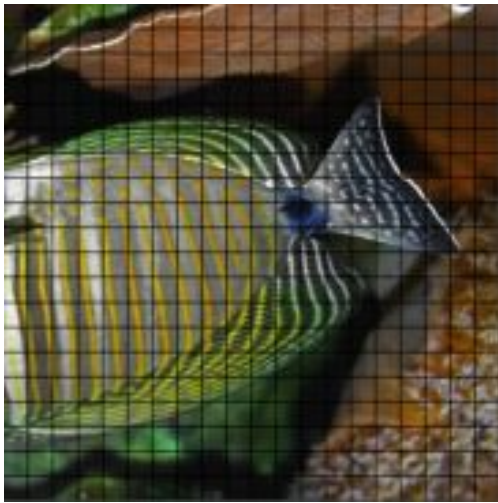
Cele mai multe dintre puzzle-uri sunt rezolvate complet corect, cu distanța 0. În ace-



(a)



(b)



(c)



(d)



(e)



(f)

Figura 3.9: **Exemple de puzzle-uri  $20 \times 20$  rezolvate.** (a)  $Distanța = 0$ . (b)  $Distanța = 0$ . (c)  $Distanța = 0$ . (d)  $Distanța = 0$ . (e)  $Distanța = 1572$ . (f)  $Distanța = 2580$ .

lași timp, acele puzzle-uri dificile tind să aibă scoruri foarte mici, având loc un „efect de domino”, în care un număr mic de alăturări diferite de soluția finală determină poziționarea greșită a multor alte piese.

În figura 3.9 sunt prezentate câteva exemple de rezolvări de puzzle-uri cu  $20 \times 20$  de piese, ilustrative pentru performanțele algoritmului.

# Capitolul 4

## Concluzii

Lucrarea a urmărit dezvoltarea a diverse distanțe, folosite cu rolul de a discrimina între perechile de laturi ale căror piese sunt alăturate în soluția finală de cele ale căror piese nu sunt alăturate în soluția finală, împreună cu un algoritm de tip Greedy de rezolvare a unor puzzle-uri cu un număr de piese între  $10 \times 10$  și  $20 \times 20$ . În procesul de dezvoltare, au fost aplicate concepte de bază ale domeniului de vedere artificială precum filtre de netezire sau distanță la nivel de culoare.

Proiectul a presupus explorarea a diverse metode și abordări, necesitând testări, ajustări și integrări ale componentelor. Rezultatele demonstrează aplicabilitatea unei tehnici deterministe în rezolvarea unui puzzle, în care deciziile luate sunt transparente și interpretabile, spre deosebire de abordarea populară în care sunt folosite rețele neurale, ce funcționează pe principiul cutii negre și unde trăsăturile sunt învățate din date și dificil de justificat. Totodată, lucrarea ilustrează complexitatea ridicată pe care o abordarea euristică o impune, cât și neajunsurile sale.

Proiectul oferă diverse direcții de dezvoltare viitoare precum îmbunătățirea metricilor de discriminare sau rafinarea componentei de luare a deciziilor prin implementarea unui bloc de decision tree. De asemenea, prin extinderea tipului de puzzle-uri rezolvate, ca de exemplu, puzzle-uri cu forme distincte sau neregulate, lucrarea capătă aplicabilități reale în domeniul restaurării de imagini.

Lucrarea de față oferă o acuratețe medie de 92% pentru puzzle-uri de  $10 \times 10$  de piese, 85% pentru puzzle-uri de  $15 \times 15$  și 82% pentru puzzle-uri de  $20 \times 20$  piese, rotația corectă fiind necunoscută. Rezolvarea puzzle-urilor cu piese de formă pătrată în care nu este cunoscută orientarea rămâne o temă activă de cercetare, eforturile prezente concentrându-se pe folosirea rețelelor neurale. Până în 2010, cele mai bune rezultate în folosirea algoritmilor Greedy au ajuns la 432 de piese cu orientări cunoscute, în prezent atingându-se performanța rezolvării a peste 9000 de piese cu orientare necunoscută.

# Bibliografie

- [1] Naif Alajlan, „Solving Square Jigsaw Puzzles Using Dynamic Programming and the Hungarian Procedure”, în *American Journal of Applied Sciences* (2009), URL: <https://thescipub.com/pdf/ajassp.2009.1941.1947.pdf>.
- [2] Taeg Sang Cho, Shai Avidan și William T. Freeman, „A Probabilistic Image Jigsaw Puzzle Solver”, în (2010), URL: <https://people.csail.mit.edu/billf/papers/JigsawSolverCVPR2010.pdf>.
- [3] Andrew C. Gallagher, „Jigsaw Puzzles with Pieces of Unknown Orientation”, în (2012), URL: [https://projet.liris.cnrs.fr/imagine/pub/proceedings/CVPR2012/data/papers/049\\_01A-04.pdf](https://projet.liris.cnrs.fr/imagine/pub/proceedings/CVPR2012/data/papers/049_01A-04.pdf).
- [4] Nadia Guerroui și Hamid Seridi, „Solving computational square jigsaw puzzles with a novel pairwise compatibility measure”, în *Journal of King Saud University - Computer and Information Sciences* (2018), URL: <https://doaj.org/article/7c327a75964f43cf8e482f258b2b1d38>.
- [5] Vahan Huroyan, Gilad Lerman și Hau-Tieng Wu, „Solving Jigsaw Puzzles by the Graph Connection Laplacian”, în *SIAM Journal on Imaging Sciences* (2020), URL: <https://doi.org/10.1137/19M1290760>.
- [6] Kilho Son, James Hays și David B. Cooper, „Solving Square Jigsaw Puzzles with Loop Constraints”, în (2014), URL: [https://faculty.cc.gatech.edu/~hays/papers/puzzle\\_eccv14.pdf](https://faculty.cc.gatech.edu/~hays/papers/puzzle_eccv14.pdf).